

[WHS4] DevSecOps Infrastructure 16반 김민서 (3625)

01. Repository

과제에서 사용한 취약한 인프라 코드와 수정 결과는 다음 저장소에서 관리하였다.

```
https://github.com/Opal1031/WHS_4th_DevSecOps_Infrastructure
```

관련 파일의 위치는 다음과 같다.

```
lab/homework/  
├── ASSIGNMENT.md  
├── vulnerable/  
│   └── main.tf  
└── fixed/  
    └── main.tf
```

02. 과제 개요

이번 과제의 목표는 Terraform으로 작성된 사내 웹서비스 인프라를 tfsec로 스캔하고, 위험한 설정을 직접 수정한 뒤 다시 스캔하여 **CRITICAL**과 **HIGH** 취약점을 모두 **0**으로 만드는 것이다.

제공된 코드는 로그 저장용 S3 버킷, 웹·DB 보안 그룹, 사용자 데이터베이스 RDS, 첨부파일 저장용 EBS 볼륨으로 구성되어 있다. 초기 코드에는 인터넷 공개, 평문 비밀번호, 저장 데이터 미암호화와 같은 보안 문제가 포함되어 있었다.

진행 순서는 다음과 같다.

1. 취약한 **main.tf**를 tfsec로 스캔
2. 탐지된 CRITICAL·HIGH 항목 확인
3. 원본 코드의 전체적인 형식을 유지하면서 보안 설정 수정
4. 수정된 **main.tf** 재스캔
5. 수정 전·후 결과 비교 및 수정 이유 정리

03. 실습 환경

구분	내용
운영체제	macOS
컨테이너 환경	Docker Desktop
IaC 언어	Terraform HCL
보안 스캐너	tfsec v1.28.14
스캐너 실행 방식	Docker 컨테이너

구분	내용
대상 클라우드	AWS
리전	ap-northeast-2

tfsec를 로컬에 직접 설치하지 않고, 저장소에 제공된 **scan.sh**를 사용해 Docker 컨테이너로 실행하였다.

```
cd lab
./scan.sh tfsec homework/vulnerable
```

04. 수정 전 스캔 결과

취약한 원본 코드를 스캔한 결과는 다음과 같다.

```
▶ 도구: tfsec    ▶ 대상: homework/vulnerable

results
-----
passed           2
ignored          0
critical         4
high            10
medium           5
low              5

2 passed, 24 potential problem(s) detected.
```

심각도	개수
CRITICAL	4
HIGH	10
MEDIUM	5
LOW	5
PASS	2

CRITICAL은 외부에 공개된 보안 그룹 규칙과 RDS에서 탐지되었다. HIGH는 S3 퍼블릭 접근, S3·RDS·EBS 미암호화 설정에서 주로 탐지되었다.

주요 탐지 항목

심각도	대상	문제
CRITICAL	Security Group	SSH 22번 포트가 0.0.0.0/0에 공개됨
CRITICAL	Security Group	MySQL 3306번 포트가 0.0.0.0/0에 공개됨

심각도	대상	문제
CRITICAL	Security Group	모든 인터넷 주소로 아웃바운드 통신이 허용됨
CRITICAL	RDS	데이터베이스가 인터넷에 공개됨
HIGH	S3	퍼블릭 ACL과 정책을 차단하는 설정이 없음
HIGH	S3	버킷 ACL이 public-read 로 설정됨
HIGH	S3	버킷 암호화와 고객 관리형 키가 없음
HIGH	RDS	스토리지 암호화가 적용되지 않음
HIGH	EBS	볼륨 암호화가 적용되지 않음

05. 취약 설정 분석 및 수정

S3 로그 버킷 공개 차단

초기 코드의 S3 버킷 ACL은 **public-read**로 설정되어 있었다.

```
resource "aws_s3_bucket_acl" "logs_acl" {
  bucket = aws_s3_bucket.logs.id
  acl    = "public-read"
}
```

로그에는 사용자 접근 기록, 시스템 동작 정보와 같은 민감한 내용이 포함될 수 있다. 버킷이 공개되면 누구나 로그를 조회할 수 있으므로 ACL을 **private**으로 변경하였다.

또한 ACL이나 버킷 정책이 실수로 변경되더라도 인터넷에 공개되지 않도록 퍼블릭 액세스 차단 옵션 네 가지를 모두 활성화하였다.

tfsec의 **aws-s3-block-public-acls** 규칙은 공개 ACL이 포함된 PUT 요청을 차단하도록 **block_public_acls = true**를 설정할 것을 권고한다. 이와 함께 공개 정책, 기존 공개 ACL 무시, 공개 버킷 제한과 관련된 tfsec 규칙을 모두 충족하도록 네 가지 설정을 적용하였다.

```
resource "aws_s3_bucket_public_access_block" "logs" {
  bucket = aws_s3_bucket.logs.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}
```

저장 데이터 암호화

초기 S3, RDS, EBS에는 암호화 설정이 없었다. 저장 장치나 스냅샷이 유출될 경우 데이터가 평문으로 노출될 수 있으므로 고객 관리형 KMS 키를 추가하였다.

```
resource "aws_kms_key" "infrastructure" {
  description      = "KMS key for company web infrastructure"
  enable_key_rotation = true
}
```

하나의 고객 관리형 KMS 키를 S3, RDS, EBS 암호화에 연결하였다. 키 자동 순환도 활성화하여 동일한 암호화 키를 장기간 사용하는 위험을 줄였다. 이는 tfsec의 S3 버킷 암호화, 고객 관리형 키 사용, RDS-EBS 저장 데이터 암호화 규칙의 권고를 반영한 것이다.

SSH와 MySQL 접근 범위 제한

초기 보안 그룹은 SSH 22번 포트와 MySQL 3306번 포트를 0.0.0.0/0에 허용하였다. 이는 인터넷의 모든 사용자가 해당 포트에 접근할 수 있다는 의미이다.

접근 가능한 네트워크를 사내망으로 제한하기 위해 `trusted_cidr` 변수를 추가하고 기본값을 10.0.0.0/8로 설정하였다.

```
variable "trusted_cidr" {
  type      = string
  default   = "10.0.0.0/8"
}
```

SSH, MySQL, 아웃바운드 규칙의 CIDR을 모두 이 변수로 변경하였다.

```
cidr_blocks = [var.trusted_cidr]
```

tfsec의 `aws-ec2-no-public-ingress-sgr` 규칙은 /0으로 인터넷 전체에 포트를 공개하지 말고, 실제 접근이 필요한 IP 주소나 범위로 제한하도록 안내한다.

실제 환경에서는 임의의 사설 대역 전체가 아니라 회사 VPN, 관리용 네트워크 또는 필요한 애플리케이션 보안 그룹으로 더 좁게 제한해야 한다.

DB 비밀번호 하드코딩 제거

초기 코드에는 데이터베이스 관리자 비밀번호가 평문으로 작성되어 있었다.

```
password = "admin1234!"
```

소스코드에 비밀번호를 작성하면 Git 기록을 통해 자격 증명이 노출될 수 있다. 따라서 비밀번호를 `sensitive` 변수로 분리하였다.

```
variable "db_password" {
  type      = string
```

```
sensitive = true
}

password = var.db_password
```

sensitive는 Terraform 출력에서 값을 숨기기 위한 설정이다. 실제 운영에서는 환경 변수, AWS Secrets Manager 또는 안전한 CI/CD Secret을 통해 값을 전달해야 한다.

RDS 인터넷 공개 차단

초기 RDS는 **publicly_accessible = true**로 설정되어 있어 인터넷에서 직접 접근할 수 있었다.

```
publicly_accessible = false
```

RDS를 비공개로 변경하고 수정한 보안 그룹을 연결하였다. 또한 **storage_encrypted**와 KMS 키를 지정해 데이터베이스 스토리지와 스냅샷을 암호화하였다.

EBS 볼륨 암호화

첨부파일용 EBS 볼륨에는 암호화 설정이 없었다. **encrypted = true**와 고객 관리형 KMS 키를 추가하여 저장 데이터를 보호하였다.

```
encrypted = true
kms_key_id = aws_kms_key.infrastructure.arn
```

06. 수정한 main.tf

원본의 섹션 순서와 리소스 구조는 유지하고, 필요한 보안 설정을 추가하거나 위험한 값을 변경하였다.

```
# =====
# 과제 · 취약한 인프라 (수업과 다른 새 시나리오)
# 상황: 사내 웹서비스 인프라를 급하게 만들었더니 곳곳이 위험합니다.
# 목표: 스캔해서 CRITICAL·HIGH 를 전부 0으로 만드세요.
# =====

provider "aws" {
  region = "ap-northeast-2"
}

# SSH·MySQL 접근을 허용할 사내망 대역
variable "trusted_cidr" {
  type    = string
  default = "10.0.0.0/8"
}

# DB 비밀번호를 코드에 직접 작성하지 않고 외부에서 입력
```

```

variable "db_password" {
  type      = string
  sensitive = true
}

# S3·RDS·EBS 암호화에 사용할 고객 관리형 KMS 키
resource "aws_kms_key" "infrastructure" {
  description      = "KMS key for company web infrastructure"
  enable_key_rotation = true
}

# -----
# 1) 로그 저장용 S3 버킷
# -----
resource "aws_s3_bucket" "logs" {
  bucket = "company-web-logs-2026"
}

resource "aws_s3_bucket_acl" "logs_acl" {
  bucket = aws_s3_bucket.logs.id
  acl    = "private" # 수정: 로그 버킷의 공개 ACL 제거
}

# 수정: ACL·정책을 통한 모든 퍼블릭 액세스 차단
resource "aws_s3_bucket_public_access_block" "logs" {
  bucket = aws_s3_bucket.logs.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}

# 수정: 고객 관리형 KMS 키로 로그 데이터 암호화
resource "aws_s3_bucket_server_side_encryption_configuration" "logs" {
  bucket = aws_s3_bucket.logs.id

  rule {
    apply_server_side_encryption_by_default {
      kms_master_key_id = aws_kms_key.infrastructure.arn
      sse_algorithm     = "aws:kms"
    }
  }
}

# -----
# 2) 웹 + DB 보안 그룹
# -----
resource "aws_security_group" "web" {
  name = "web-sg"

  ingress {
    description = "SSH"
    from_port   = 22
  }
}

```

```

    to_port      = 22
    protocol     = "tcp"
    cidr_blocks  = [var.trusted_cidr] # 수정: SSH 접근을 사내망으로 제한
  }

  ingress {
    description = "MySQL"
    from_port   = 3306
    to_port     = 3306
    protocol    = "tcp"
    cidr_blocks = [var.trusted_cidr] # 수정: DB 접근을 사내망으로 제한
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = [var.trusted_cidr] # 수정: 외부 송신도 사내망으로 제한
  }
}

# -----
# 3) 사용자 DB
# -----
resource "aws_db_instance" "users" {
  identifier      = "company-users"
  engine          = "mysql"
  instance_class  = "db.t3.micro"
  allocated_storage = 20
  username        = "admin"
  password        = var.db_password # 수정: 하드코딩 대신 민감 변수 사용
  skip_final_snapshot = true

  publicly_accessible = false # 수정: 인터넷에서 DB 직접 접근 차단
  storage_encrypted    = true  # 수정: RDS 저장 데이터 암호화
  kms_key_id           = aws_kms_key.infrastructure.arn
  vpc_security_group_ids = [aws_security_group.web.id]
}

# -----
# 4) 첨부파일용 EBS 볼륨
# -----
resource "aws_ebs_volume" "attachments" {
  availability_zone = "ap-northeast-2a"
  size              = 20
  encrypted         = true # 수정: EBS 저장 데이터 암호화
  kms_key_id        = aws_kms_key.infrastructure.arn
}

```

07. 수정 후 재스캔

수정한 코드에 동일한 tfsec 스캔을 실행하였다.

```
cd lab
./scan.sh tfsec homework/fixed
```

실행 결과는 다음과 같다.

```
▶ 도구: tfsec    ▶ 대상: homework/fixed

results
-----
passed          19
ignored         0
critical        0
high            0
medium          5
low             3

19 passed, 8 potential problem(s) detected.
```

수정 전·후 비교

심각도	수정 전	수정 후	변화
CRITICAL	4	0	-4
HIGH	10	0	-10
MEDIUM	5	5	0
LOW	5	3	-2
PASS	2	19	+17

과제의 핵심 목표인 **CRITICAL 0, HIGH 0**을 달성하였다.

수정 후 남은 항목은 S3 로깅·버저닝, RDS 백업 보존 기간·IAM 인증·삭제 방지, 보안 그룹 설명, RDS Performance Insights와 같은 MEDIUM·LOW 수준의 추가 하드닝 권고이다.

08. 결론

취약한 Terraform 코드를 tfsec로 스캔하고, 탐지 결과를 기준으로 S3, Security Group, RDS, EBS 설정을 수정하였다. 수정 전에는 CRITICAL 4건, HIGH 10건이 탐지되었으나 퍼블릭 접근 차단, 네트워크 범위 제한, 비밀번호 변수화, KMS 기반 암호화를 적용한 뒤 CRITICAL과 HIGH가 모두 0으로 감소한 것을 확인하였다.

이번 과제를 통해 IaC는 인프라를 빠르게 구성할 수 있지만, 공개 범위나 암호화와 같은 설정 하나가 전체 서비스의 보안 수준에 직접적인 영향을 준다는 것을 확인하였다. 또한 코드 작성 후 스캐너를 한 번 실행하는 것으로 끝내지 않고, 탐지 결과를 수정한 뒤 다시 검증하는 과정이 중요하다는 점을 알 수 있었다.

한 줄 회고: 가장 위험하다고 느낀 항목은 MySQL 포트와 RDS를 인터넷에 직접 공개한 설정으로, 단순한 설정 한 줄이 중요 데이터 전체를 외부 공격에 노출할 수 있다는 점을 확인했다.

09. 참고자료

수정 방향은 수정 전 tfsec 결과에 표시된 규칙 ID와 tfsec v1.28.13 공식 문서의 Suggested Resolution 및 Secure Example을 기준으로 정리하였다.

S3

- S3 Access block should block public ACL
- S3 Access block should block public policy
- S3 Access block should ignore public ACL
- S3 Access block should restrict public bucket
- S3 Bucket does not have encryption enabled
- S3 encryption should use a customer managed key
- S3 Bucket has a public ACL

Security Group

- Ingress security group rule allows traffic from /0
- Egress security group rule allows traffic to /0

RDS

- RDS instance is exposed publicly
- RDS instance storage is not encrypted

EBS·KMS

- EBS volume is not encrypted
- EBS volume should use a customer managed key
- KMS key should have automatic rotation enabled